UNIVERSIDAD NACIONAL DE SAN AGUSTIN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



JESUS MARTIN SILVA FERNANDEZ
M en C CIENCIAS DE LA COMPUTACION

GUÍA DE LABORATORIO

CALIDAD DE SOFTWARE

SEMESTRE 8

COMPETENCIAS

Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustada a los recursos disponibles del cliente.. Cm

Laboratorio 7

Pruebas Unitarias

I

OBJETIVOS

Conocer, evaluar y utilizar efectivamente metodologías y herramientas para aplicar prueba unitaria a componente código de software

II

TEMAS A TRATAR

- **Pruebas Unitarias**
- **JUnit**

III

MARCO TEÓRICO

- **Pruebas Unitarias**

  Se dice que un programa es aceptable cuando:
  - hace lo que debe hacer
  - no hace lo que no debe hacer

  Un programador jamás debería entregar un programa sin haberlo probado. Quien recibe un programa de otro, jamás debería aceptarlo sin haberlo probado. A partir de la especificación de lo que se espera de un programa, se prepara una batería de casos de prueba.

  La batería que prepara el programador le permite saber cuándo ha terminado su trabajo. La batería que prepara quien va a recibir el programa le permite saber si puede aceptarlo o no. Los casos de prueba se pueden escribir en papel y ejecutarlos disciplinadamente de forma manual; pero esta estrategia sólo es viable para programas pequeños. En programación profesional conviene automatizar las pruebas de forma que se pueden realizar muchas veces, tanto durante el desarrollo (hasta estar satisfechos), como durante la aceptación (para dar el programa por correcto, como si en el futuro hay que modificar el programa (para cerciorarnos de que no hemos roto nada que antes funcionaba).

- **JUnit**

JUnit es un paquete Java para automatizar las pruebas de clases Java. Se puede descargar de http://junit.org/ aunque debe decirse que viene ya instalado en los entornos de desarrollo habituales. Usaremos la versión 4. Los ejemplos de este documento se han realizado con junit 4.12. Una batería de pruebas es una clase java. import static org.junit.Assert.*; import org.junit.Test; public class Ejemplo { @Test public void test() { // prueba } } La clase puede incluir sus variables privadas y el constructor correspondiente.

ACTIVIDADES

1. Procedimiento

   - Ejecutar ejercicios resueltos y propuestos
   - Escriba un reporte sobre las tareas realizadas y resultados.
   - Escriba sus conclusiones

2. Agenda:

Instalación y pruebas	90 minutos
Generación de reporte	15 minutos

3. Estructura de reporte

   I. Objetivo
   II. Descripción del procedimiento realizado en la práctica
   III. Resultados obtenidos
   IV. Análisis de resultados
   V. Conclusiones

El envío al drive compartido del reporte, incluye código generado por alumno.

V

EJERCICIOS RESUELTOS

1. Instalar adecuadamente herramienta JUnit, Cactus, testng.
   https://github.com/junit-team/junit5/
   https://jakarta.apache.org/cactus/downloads.html

VI

EJERCICIOS PROPUESTOS

1. Implementar actividades de generación de casos de prueba utilizando junit_tutorial adjunto y, el código del Anexo 1
2. Probar funcionalidad y alcances mas importantes.
3. Evaluar, verificar y validar resultados

VII

CUESTIONARIO

a) Que tipo de prueba se realiza con las pruebas unitarias, que aspecto de calidad mide?
b) Las pruebas unitarias son obligatorias, porque?
c) Que modelos de calidad utilizan los resultados de pruebas unitarias?

VIII

BIBLIOGRAFÍA

- http://www.dit.upm.es/~pepe/doc/adsw/base/junit.pdf
- https://www.uv.mx/personal/jfernandez/files/2010/07/Uso-JUnit.pdf

ANEXO 1

```
// JUnit test example

public class Rectangle
{
      // State variables.
      private int height;
      private int width;

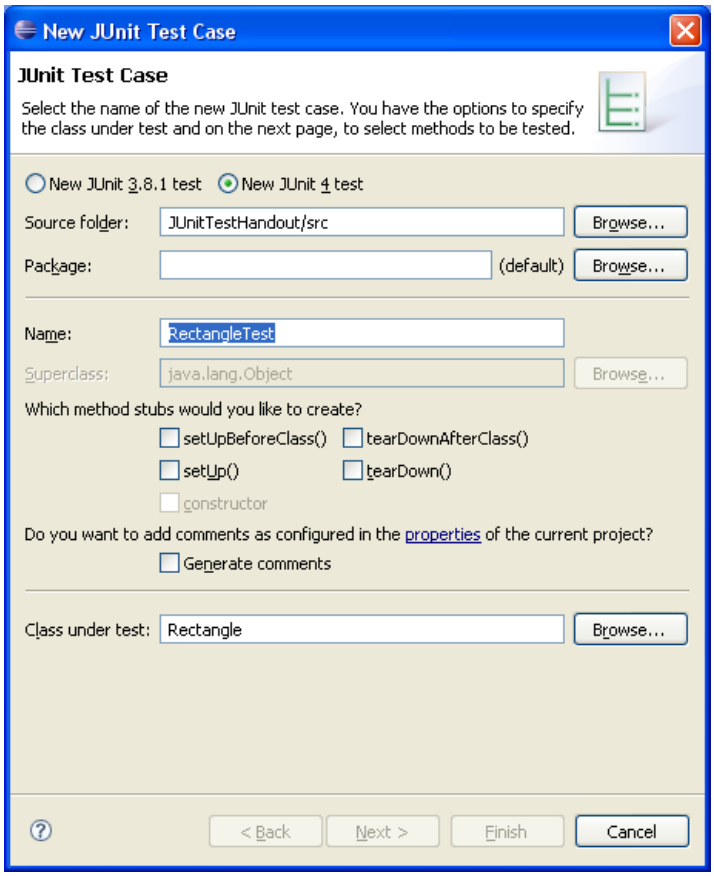
      // Constructors
      Rectangle()
      {
            height=0;
            width =0;
      }
      Rectangle(int h,int w)
      {
            height=h;
            width =w;
      }

      // Readers
      public int getH()
            {return height;}
      public int getW()
            {return width;}
      public int getA()
            {return height* width;}

      // Writers
      public void setH(int h)
            {height=h;}
      public void setW(int w)
            {width =w;}

      public String toString()
            {return "Rectangle: height="+height+"; width="+ width +
            "; area="+getA()+".";}

}  // End of Rectangle class
```


New JUnit Test Case
JUnit Test Case
Select the name of the new JUnit test case. You have the options to specify the class under test and on the next page, to select methods to be tested.
New JUnit 3.8.1 test
New JUnit 4 test
Source folder:
JUnitTestHandout/src
Browse...
Package:
(default)
Browse...
Name:
RectangleTest
Superclass:
java.lang.Object
Browse...
Which method stubs would you like to create?
setUpBeforeClass()
tearDownAfterClass()
setUp()
tearDown()
constructor
Do you want to add comments as configured in the properties of the current project?
Generate comments
Class under test:
Rectangle
Browse...
< Back
Next >
Finish
Cancel

```
import static org.junit.Assert.*;
import org.junit.*;

public class RectangleTest
{
	Rectangle r;
	Rectangle[] rList=new Rectangle[5];

	@Before
	// Will be performed before each test.
	public void testSetup()
	{
		System.out.println("Setup for test complete.");
	}

	@After
	// Will be performed after each test.
	public void testComplete()
	{
		System.out.println("Test complete.");
	}

	@Test
	public void test1()
	{
		// Can be run when reader methods and default
		// constructor are complete.
		r=new Rectangle();
		try
		{
			assertTrue("Test 1: Default values are wrong.",
				r.getH()==0 && r.getW()==0);
			System.out.println("Test 1 completed successfully.");
		}
		catch (AssertionError e)
		{
			System.out.println(e.getMessage());
		}
	}

	@Test
	public void test2()
	{
		// After area reader is written.
		r=new Rectangle();
		try
		{
			assertTrue("Test 2: Default values are wrong.",
				r.getH()==0 && r.getW()==0 && r.getA()==0);
			System.out.println("Test 2 completed successfully.");
		}
		catch (AssertionError e)
		{
			System.out.println(e.getMessage());
		}
	}
```

```
@Test
public void test3()
{
	// After second constructor is written.
	r=new Rectangle();
	try
	{
		r=new Rectangle(10,20);
		assertTrue("Test 3: Initial values are wrong.",
			r.getH()==10 && r.getW()==20 && r.getA()==200);
		System.out.println("Test 3 completed successfully.");
	}
	catch (AssertionError e)
	{
		System.out.println(e.getMessage());
	}
}

@Test
public void test4()
{
	// After second constructor is written.
	r=new Rectangle();
	try
	{
		for (int i=0;i<5;i++)
		{
			rList[i] = new Rectangle(2*i,3*i);
			assertTrue("Test 4: Initial values are wrong.",
				rList[i].getH()==2*i &&
				rList[i].getW()==3*i &&
				rList[i].getA()==6*i*i);
		}
		System.out.println("Test 4 completed successfully.");
	}
	catch (AssertionError e)
	{
		System.out.println(e.getMessage());
	}
}

@Test
public void test5()
{
	// After writers are written.
	r=new Rectangle();
	try
	{
		for (int i=0;i<5;i++)
		{
			rList[i] = new Rectangle(2*i,3*i);
			rList[i].setH(11*i);
			rList[i].setW(7*i);
			assertTrue("Test 5: Modified values are wrong.",
				rList[i].getH()==11*i &&
				rList[i].getW()==7*i &&
				rList[i].getA()==77*i*i);
		}
		System.out.println("Test 5 completed successfully.");
```

```
		}
		catch (AssertionError e)
		{
			System.out.println(e.getMessage());
		}
	}

	@Test
	public void test6()
	{
		// toString test.
		r=new Rectangle();
		try
		{
			for (int i=0;i<5;i++)
			{
				rList[i] = new Rectangle(2*i,3*i);
				rList[i].setH(11*i);
				rList[i].setW(7*i);
				assertTrue("Test 6: Modified values are wrong.",
					rList[i].getH()==11*i &&
					rList[i].getW()==7*i &&
					rList[i].getA()==77*i*i);
				System.out.println(rList[i]);
			}
			System.out.println("Test 6 completed successfully.");
		}
		catch (AssertionError e)
		{
			System.out.println(e.getMessage());
		}
	}

	public static void main(String[] args)
	{
		org.junit.runner.JUnitCore.main("RectangleTest");
	}
}
```

```
Output from correct Rectangle class test.

Setup for test complete.
Test 1 completed successfully.
Test complete.
Setup for test complete.
Test 2 completed successfully.
Test complete.
Setup for test complete.
Test 3 completed successfully.
Test complete.
Setup for test complete.
Test 4 completed successfully.
Test complete.
Setup for test complete.
Test 5 completed successfully.
Test complete.
Setup for test complete.
Rectangle: height=0; width=0; area=0.
Rectangle: height=11; width=7; area=77.
Rectangle: height=22; width=14; area=308.
Rectangle: height=33; width=21; area=693.
Rectangle: height=44; width=28; area=1232.
Test 6 completed successfully.
Test complete.


I changed setH() so that height was set to h*2 and reran the tests.

Setup for test complete.
Test 1 completed successfully.
Test complete.
Setup for test complete.
Test 2 completed successfully.
Test complete.
Setup for test complete.
Test 3 completed successfully.
Test complete.
Setup for test complete.
Test 4 completed successfully.
Test complete.
Setup for test complete.
Test 5: Modified values are wrong.
Test complete.
Setup for test complete.
Rectangle: height=0; width=0; area=0.
Test 6: Modified values are wrong.
Test complete.
```